Bitácora de Desarrollo

Laboratorio Big Data - Arquitectura **Data Lakehouse**

Análisis de publicaciones en redes sociales (Instagram, TikTok y X)
durante la semana de Fiestas Patrias 2026

Dato	Detalle
Arquitectura	Data Lakehouse (zonas RAW, CLEAN y PREPARED)
Tecnologías	Python 3.11, PySpark 3.5.5, Java 17, Parquet, pandas, matplotlib
Sistema operativo	Windows 11 (entorno conda *lakehouse*)
Fechas de trabajo	29-09-2026 y 30-09-2026
Exposición	06-10-2026
Rol de esta bitácora	Desarrollo del pipeline (código, ejecución y evidencias)
Integrantes	_____

Esta bitácora explica paso a paso cómo se construyó el pipeline, qué hace cada script, qué problemas aparecieron y cómo se resolvieron. Está pensada para que todo el equipo pueda entender el trabajo, reutilizar las capturas en el informe y defender la demo en la exposición.

Contenido

- 1. Resumen del proyecto
- 2. ¿Qué es un Data Lakehouse? (explicado simple)
- 3. Arquitectura de la solución
- 4. Herramientas y estructura de carpetas
- 5. Registro cronológico (bitácora)
- 6. Paso 0 - Preparación del entorno
- 7. Problema 1 - Spark en Windows (winutils)
- 8. Paso 1 - Ingesta a la zona RAW
- 9. Paso 2 - Limpieza con PySpark (zona CLEAN)
- 10. Paso 3 - Zona PREPARED
- 11. Paso 4 - Consultas y gráficos
- 12. Resultados y hallazgos
- 13. Problemas encontrados y soluciones
- 14. Recomendaciones y tips
- 15. Cómo reproducir todo (guía rápida)
- 16. Preguntas probables en la exposición
- 17. Glosario
- 18. Índice de evidencias

1. Resumen del proyecto

Se construyó una **pequeña solución Big Data simulada en Windows** con la arquitectura **Data Lakehouse**. El caso elegido es el análisis de publicaciones en tres redes sociales (Instagram, TikTok y X) durante la semana del 14 al 20 de septiembre de 2026, que incluye las Fiestas Patrias de Chile.

Pregunta que responde el proyecto: ¿qué plataforma, qué hashtags, qué horario y qué tipo de contenido generan más interacción (engagement)?

Etapa	Qué se hizo	Resultado
Ingesta	Se simuló una API que entrega publicaciones en páginas JSON y un CSV de usuarios, con errores a propósito.	8.237 publicaciones y 300 usuarios en RAW
Limpieza	PySpark corrige errores, quita duplicados, crea columnas nuevas y guarda en Parquet.	7.674 publicaciones limpias (563 eliminadas)
Zona preparada	Se crearon 5 tablas resumen listas para analizar.	5 tablas Parquet
Consultas	Consultas con Spark SQL y pandas, 4 gráficos y reportes CSV.	Respuestas al análisis y visualizaciones

2. ¿Qué es un Data Lakehouse? (explicado simple)

Un **Data Lake** es un lugar donde se guardan archivos de cualquier tipo, baratos y en bruto (como una gran bodega). Un **Data Warehouse** es una base de datos ordenada y optimizada para hacer consultas SQL (como un supermercado con todo en su estante). El **Data Lakehouse** junta lo mejor de ambos: guarda los datos como archivos (barato y flexible), pero en un formato optimizado (Parquet) que permite consultarlos con SQL como si fuera una base de datos.

Normalmente se organiza en **zonas o capas** por las que los datos van "subiendo de calidad":

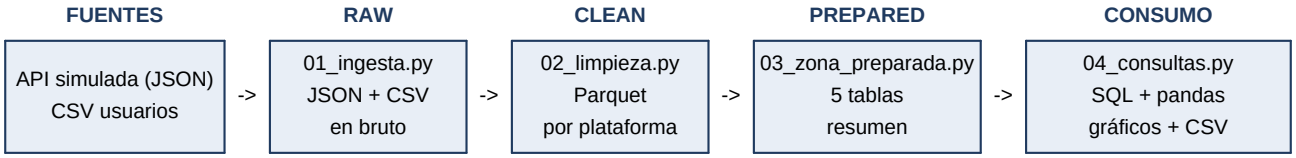
Zona	También llamada	Qué contiene	En nuestro proyecto
RAW	Bronze / bruto	Los datos tal como llegan, con errores	data/raw (JSON y CSV)
CLEAN	Silver / limpio	Datos corregidos y con formato correcto	data/clean (Parquet)
PREPARED	Gold / preparado	Tablas resumen listas para análisis	data/prepared (Parquet)

Idea clave para la exposición

En un lakehouse real las carpetas serían buckets de Amazon S3 o directorios de HDFS. Aquí se simulan con carpetas locales de Windows, pero la lógica es la misma: los datos nunca se borran de RAW, y cada zona se puede regenerar desde la anterior.

3. Arquitectura de la solución

El flujo completo del pipeline es el siguiente (cada caja es un script de Python):



Por qué esta arquitectura y no Lambda o Kappa: nuestro caso es de **análisis histórico** (qué pasó durante una semana), no de alertas en tiempo real. No necesitamos streaming ni Kafka, por lo que el Lakehouse es más simple de implementar, más barato y permite consultar los mismos datos con SQL y con pandas.

4. Herramientas y estructura de carpetas

Herramienta	Versión	Para qué se usó
Python	3.11	Lenguaje de todos los scripts
Miniconda / conda	-	Crear un entorno aislado (*lakehouse*) con todas las librerías
Java (OpenJDK)	17	Spark funciona sobre Java; es obligatorio
PySpark	3.5.5	Procesamiento distribuido: lectura, limpieza, agregaciones y SQL
Hadoop winutils	3.3.5	Archivos que Spark necesita en Windows para leer/escribir disco
Parquet	-	Formato columnar y comprimido para las zonas CLEAN y PREPARED
pandas + pyarrow	3.0.6	Leer Parquet y hacer consultas rápidas
matplotlib	-	Gráficos
VS Code	-	Editor de código

Estructura del proyecto

```
Lakehouse_RedesSociales/
|-- config.py                rutas de las zonas y funcion crear_spark()
|-- README.md                instrucciones de instalacion y ejecucion
|-- scripts/
|   |-- 01_ingesta.py        genera los datos en bruto (zona RAW)
|   |-- 02_limpieza.py       limpia con PySpark y guarda Parquet (zona CLEAN)
|   |-- 03_zona_preparada.py crea 5 tablas resumen (zona PREPARED)
|   |-- 04_consultas.py      Spark SQL, pandas, graficos y reportes
|-- data/
|   |-- raw/publicaciones/fecha=2026-09-14 ... fecha=2026-09-20/  (JSON)
|   |-- raw/usuarios/usuarios.csv
|   |-- clean/publicaciones/plataforma=Instagram|TikTok|X/  (Parquet)
|   |-- prepared/resumen_diario, top_hashtags, ranking_usuarios,
|               actividad_hora, tipo_contenido  (Parquet)
|-- salida/                  4 graficos PNG + 2 reportes CSV
|-- evidencias/              capturas de pantalla de cada paso
```

config.py centraliza las rutas (RAW, CLEAN, PREPARED, SALIDA) y la función `crear_spark()`, así ningún script repite esa configuración. Esto aporta modularidad: si cambia una ruta, se cambia en un solo lugar.

5. Registro cronológico (bitácora)

Fecha	Actividad	Resultado
29-09	Definición del tema (redes sociales) y división del trabajo del equipo.	Rol de desarrollo asignado.
29-09	Creación del entorno conda *lakehouse* con Python 3.11, Java 17 y PySpark 3.5.	Entorno instalado y verificado.
29-09	Creación de config.py y del script de ingesta (01).	8.237 publicaciones en RAW.
29-09	Primera ejecución de la limpieza (02).	**Error:** UnsatisfiedLinkError de Hadoop en Windows.
29-09	Instalación de winutils.exe y hadoop.dll, variable HADOOP_HOME y Path.	Error resuelto. Spark lee y escribe Parquet.
29-09	Ejecución de la limpieza y capturas de evidencia.	7.674 publicaciones limpias.
30-09	Recuperación de 01_ingesta.py (borrado por error) desde la Papelera.	Archivo restaurado sin pérdida.
30-09	Construcción incremental de 03_zona_preparada.py (tabla por tabla).	5 tablas en data/prepared.
30-09	Ajuste de comentarios del código (criterio de calidad).	Código documentado.
30-09	Construcción de 04_consultas.py: Spark SQL, pandas, gráficos y CSV.	4 gráficos y 2 reportes en salida/.
30-09	Organización de 27 evidencias y redacción de esta bitácora.	Material listo para el informe.

6. Paso 0 - Preparación del entorno

Se usó **conda** para crear un entorno aislado. Un entorno **no es una máquina virtual**: es una carpeta con su propio Python y librerías, dentro del mismo Windows. Así no se mezclan versiones con otros proyectos.

Comandos (en Anaconda Prompt)

```
conda create -y -n lakehouse -c conda-forge python=3.11 openjdk=17
            pyspark=3.5 pandas pyarrow matplotlib      (todo en una sola linea)
conda activate lakehouse
python --version
java -version
```

Importante para el equipo

Cada vez que se abre Anaconda Prompt hay que ejecutar `conda activate lakehouse`. Al inicio de la línea debe aparecer **(lakehouse)**. Si no, aparece el error `ModuleNotFoundError: No module named 'pyspark'`.

7. Problema 1 - Spark en Windows (winutils)

Síntoma

Al ejecutar por primera vez `02_limpieza.py`, Spark falló al leer la carpeta RAW:

```
java.lang.UnsatisfiedLinkError:
  'boolean org.apache.hadoop.io.nativeio.NativeIO$Windows.access0(java.lang.String, int)'
```

Causa

Spark usa **Hadoop** por dentro para acceder a archivos. Hadoop está pensado para Linux y en Windows necesita dos archivos nativos: **winutils.exe** y **hadoop.dll**. Apache no los distribuye compilados, por lo que no vienen con PySpark.

Solución aplicada

- Se descargaron **winutils.exe** y **hadoop.dll** de la versión **hadoop-3.3.5** desde el repositorio comunitario *github.com/cdarlint/winutils* (compatible con Hadoop 3.3.4, que es el que trae Spark 3.5.5). La versión 2.6.1 **no** sirve.
- Se copiaron en `C:\hadoop\bin`.
- Se creó la variable de entorno de usuario `HADOOP_HOME = C:\hadoop` (por ventana y con el comando `setx`).
- Se agregó `C:\hadoop\bin` al **Path** del usuario.
- Se cerró y volvió a abrir Anaconda Prompt para cargar las variables.

```
setx HADOOP_HOME "C:\hadoop"
echo %HADOOP_HOME%        (debe responder C:\hadoop)
winutils.exe              (debe mostrar la ayuda "Usage: ...")
```

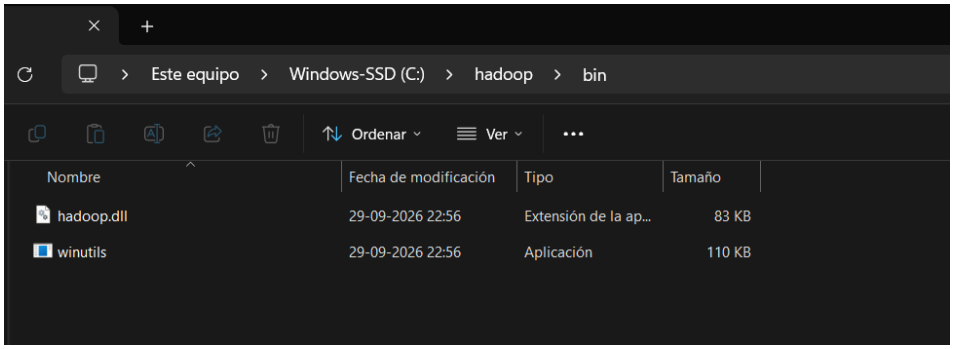


Figura 1. Carpeta C:\hadoop\bin con winutils.exe y hadoop.dll.

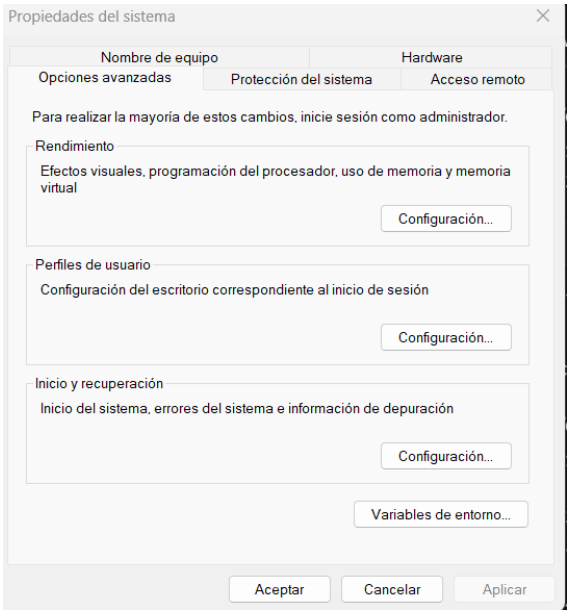


Figura 2. Propiedades del sistema - botón Variables de entorno.

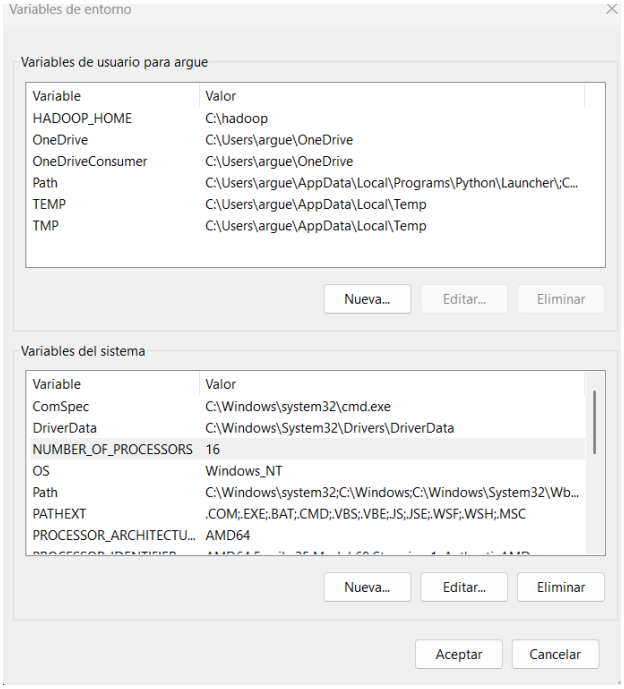


Figura 3. Variable de usuario HADOOP_HOME = C:\hadoop creada.

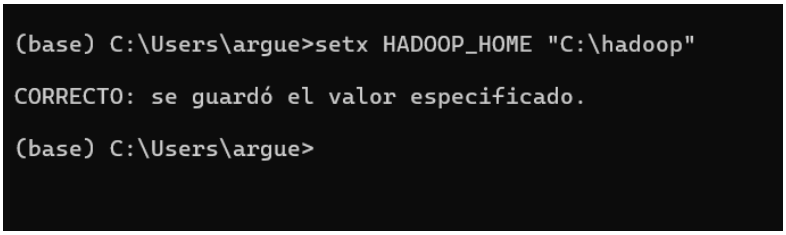


Figura 4. Creación de la variable con el comando setx.

Nota sobre seguridad

winutils proviene de un repositorio comunitario, no oficial de Apache. Es el método más usado para correr Spark en Windows, pero conviene mencionarlo. En un entorno real se usaría Linux, Docker o un servicio en la nube, donde este problema no existe.

8. Paso 1 - Ingesta a la zona RAW

Script: `scripts/01_ingesta.py`. Simula dos fuentes de datos, como ocurriría en una empresa real:

- **Una API de redes sociales** que entrega las publicaciones en **páginas JSON de 100 registros** (así funcionan las APIs reales de Instagram, TikTok o X).
- **Un archivo CSV** exportado con los 300 usuarios (id, nombre, país, fecha de registro, seguidores).

Decisiones de diseño de los datos

Parámetro	Valor	Por qué
Semilla aleatoria	random.seed(42)	Siempre se generan los mismos datos (resultados reproducibles).
Período	14 al 20 de septiembre 2026	Incluye Fiestas Patrias: permite ver un peak de actividad.
Volumen	800 publicaciones/día; 1.600 los días 17, 18 y 19	Simula el aumento de actividad del "18".
Horas	Pesos por hora (más de 19 a 21 h)	Imita el uso real de redes: más actividad en la noche.
Métricas	Vistas según plataforma; likes = 2% a 12% de las vistas	TikTok tiene más vistas que Instagram, e Instagram más que X.
Hashtags	12 generales + 5 de Fiestas Patrias	Los de Fiestas Patrias solo aparecen del 17 al 19.

Errores introducidos a propósito (para que la limpieza tenga sentido)

Error	Ejemplo	Cantidad en RAW
Plataforma mal escrita	"IG", "INSTAGRAM", " Instagram ", "Tik Tok", "twitter"	13 formas distintas de escribir 3 redes
Publicaciones duplicadas	La API reenvía la misma publicación	237 duplicados
Usuario nulo	"user_id": null	181
Fecha inválida	"fecha_hora": "fecha_invalida"	163
Likes negativos	"likes": -348	147
Comentarios faltantes	"comentarios": null	147
País inconsistente (CSV)	"Chile", "chile", "CL"	Varios usuarios

Particionado por fecha

Cada día se guarda en su propia carpeta: `raw/publicaciones/fecha=2026-09-14/`. Esta convención `columna=valor` es la que usan S3 y HDFS y Spark la reconoce automáticamente.

```
python scripts/01_ingesta.py
```

```
2026-09-14 -> 9 paginas JSON
2026-09-15 -> 9 paginas JSON
2026-09-16 -> 9 paginas JSON
2026-09-17 -> 17 paginas JSON
2026-09-18 -> 17 paginas JSON
2026-09-19 -> 17 paginas JSON
2026-09-20 -> 9 paginas JSON
Publicaciones guardadas en bruto: 8237
```

Figura 5. Ejecución de la ingesta: 300 usuarios y 8.237 publicaciones. Los días 17 a 19 tienen 17 páginas en vez de 9.

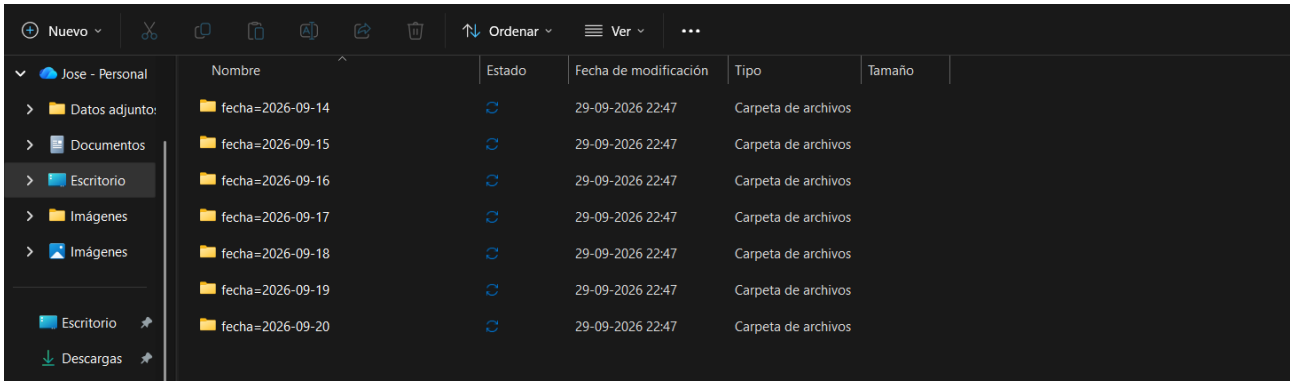


Figura 6. Zona RAW particionada por fecha (una carpeta por día).



Figura 7. JSON en bruto (formateado en VS Code para leerlo): la publicación P7 tiene "fecha_invalida" (4 casos en esta página).

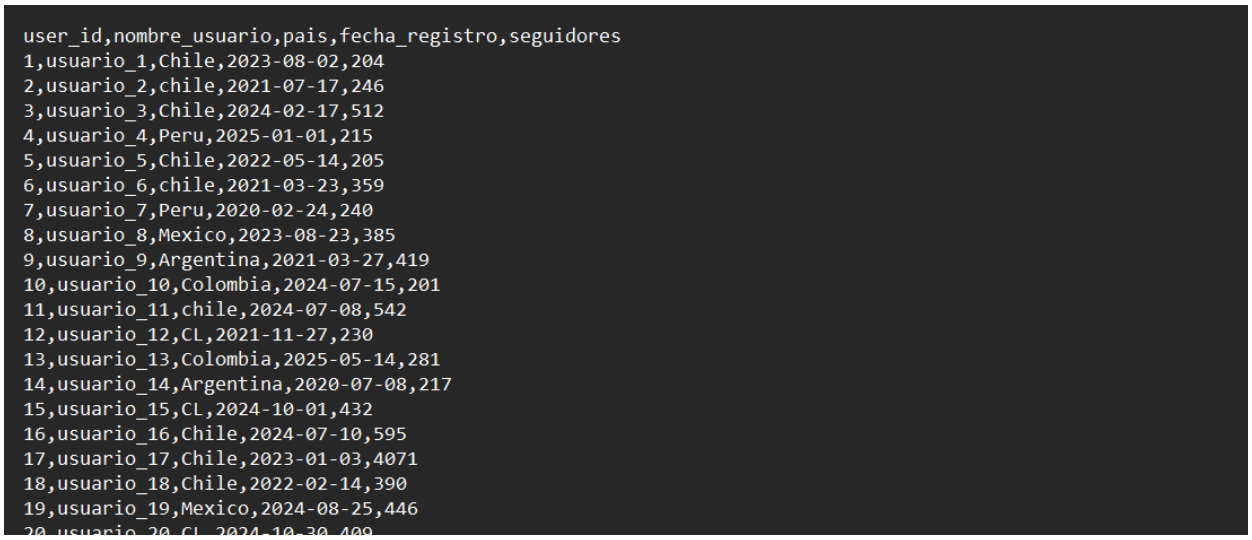


Figura 8. CSV de usuarios: el país viene escrito como "Chile", "chile" y "CL".

9. Paso 2 - Limpieza con PySpark (zona CLEAN)

Script: `scripts/02_limpieza.py`. Lee la zona RAW con Spark, corrige los errores, crea columnas nuevas, cruza con los usuarios y guarda en **Parquet**.

Transformaciones aplicadas (en orden)

#	Transformación	Función de PySpark	Para qué
1	Leer las páginas JSON y separar las publicaciones	spark.read.json() + F.explode("datos")	Cada página trae una lista; explode deja 1 fila por publicación.
2	Unificar nombre de plataforma	F.lower(F.trim()) + F.when()	13 variantes -> Instagram, TikTok, X.
3	Quitar publicaciones sin usuario	filter(isNotNull)	No se pueden asociar a nadie.
4	Quitar duplicados	dropDuplicates(["id_post"])	Evita contar dos veces.
5	Convertir fecha a timestamp	F.to_timestamp()	Las fechas inválidas quedan nulas y se eliminan.
6	Nulos y negativos a 0	F.coalesce() + F.when(col < 0, 0)	Métricas consistentes.
7	Columnas fecha y hora	F.to_date(), F.hour()	Para agrupar por día y por hora.
8	Extraer hashtags del texto	regexp_extract_all(texto, '(#\w+)')	Crea una lista (array) de hashtags.
9	Engagement	likes + comentarios + compartidos	Interacción total de la publicación.
10	Tasa de engagement	engagement / vistas * 100	Interacción relativa (%), comparable entre redes.
11	Normalizar país y cruzar con usuarios	join(usuarios, "user_id", "left")	Agrega nombre, país y seguidores.
12	Guardar particionado por plataforma	write.partitionBy("plataforma").parquet()	Consultas más rápidas por red social.

Fragmento clave del código

```
plataforma = F.lower(F.trim(F.col("plataforma")))
df = df.withColumn("plataforma",
    F.when(plataforma.isin("instagram", "ig"), "Instagram")
     .when(plataforma.isin("tiktok", "tik tok"), "TikTok")
     .when(plataforma.isin("x", "twitter"), "X")
     .otherwise("Otra"))

df = df.filter(F.col("user_id").isNotNull())
df = df.dropDuplicates(["id_post"])
df = df.withColumn("fecha_hora", F.to_timestamp("fecha_hora", "yyyy-MM-dd HH:mm:ss"))
df = df.filter(F.col("fecha_hora").isNotNull())

python scripts/02_limpieza.py
```

Resultado

Indicador	Valor
Publicaciones en RAW (antes)	8.237
Publicaciones limpias (después)	7.674
Filas eliminadas	563 (duplicados, usuarios nulos y fechas inválidas; algunos registros tenían más de un error)
Plataformas	13 variantes -> 3 (Instagram 2.502, TikTok 2.547, X 2.625)
Tamaño RAW (JSON)	2,2 MB
Tamaño CLEAN (Parquet)	389 KB: unas 5,7 veces menos, aun con más columnas

```
============================================================
 PASO 2 - LIMPIEZA CON PYSPARK
============================================================
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Publicaciones en RAW (antes): 8237

Plataformas como vienen en bruto:
+------------+-----+
| plataforma|count|
+------------+-----+
|     TikTok|  721|
|    twitter|  719|
|          x|  702|
|          X|  696|
|    TIKTOK |  690|
|    Twitter|  682|
|     tiktok|  681|
|    Tik Tok|  670|
|  Instagram|  569|
| Instagram |  562|
|  instagram|  533|
|         IG|  507|
```

Figura 9. Lectura de la zona RAW con PySpark (ya con winutils funcionando): 8.237 registros y plataformas sin limpiar.

```
Publicaciones limpias (despues): 7674
Filas eliminadas: 563

Plataformas despues de limpiar:
+----------+-----+
|plataforma|count|
+----------+-----+
| Instagram| 2502|
|    TikTok| 2547|
|         X| 2625|
+----------+-----+

Ejemplo de datos limpios:
+-------+----------+-------------------+----------------------------+----------+---------------+-----+
|id_post|plataforma|fecha_hora         |hashtags                    |engagement|tasa_engagement|pais |
+-------+----------+-------------------+----------------------------+----------+---------------+-----+
|P1     |X         |2026-09-14 10:13:47|[#comida, #humor, #gaming]  |247       |5.0            |Chile|
|P10    |X         |2026-09-14 04:05:20|[#viajes, #santiago]        |165       |2.06           |Chile|
|P100   |TikTok    |2026-09-14 19:34:03|[]                          |831       |2.64           |Peru |
|P1000  |TikTok    |2026-09-15 16:48:01|[#tecnologia]               |4553      |10.75          |Chile|
|P1001  |Instagram |2026-09-15 21:16:31|[#moda, #valparaiso, #viajes]|491       |7.11           |Chile|
+-------+----------+-------------------+----------------------------+----------+---------------+-----+
only showing top 5 rows
```

Figura 10. Después de limpiar: 7.674 registros, 3 plataformas y columnas nuevas (hashtags, engagement, tasa, país).

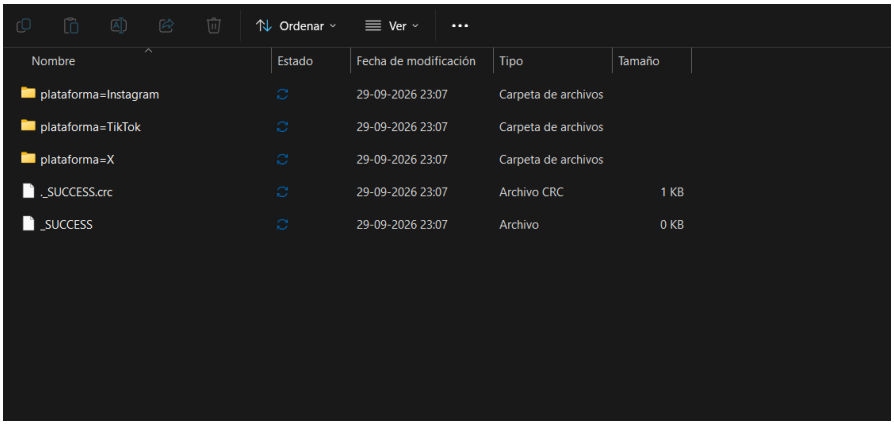


Figura 11. Zona CLEAN particionada por plataforma. _SUCCESS indica que Spark terminó de escribir sin errores.

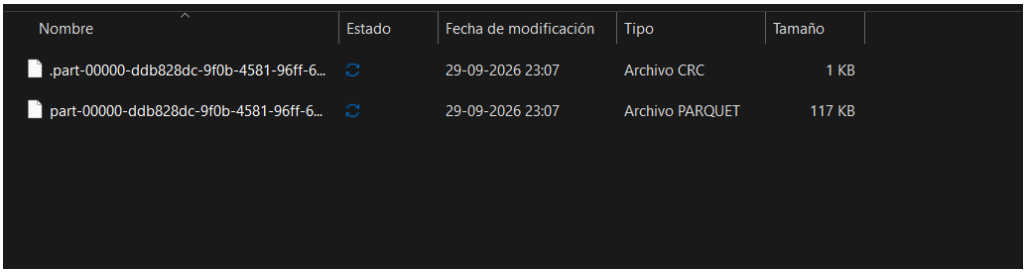


Figura 12. Archivo Parquet dentro de plataforma=Instagram (117 KB) y su archivo de verificación .crc.

10. Paso 3 - Zona PREPARED

Script: `scripts/03_zona_preparada.py`. Lee el Parquet limpio y crea **5 tablas resumen** con `groupBy().agg()`. Se construyó de forma **incremental**: se agregó una tabla, se ejecutó, se revisó el resultado y se capturó la evidencia antes de seguir con la siguiente.

Tabla	Agrupa por	Calcula	Pregunta que responde
resumen_diario	fecha, plataforma	publicaciones, vistas, engagement promedio	¿Cómo evolucionó la actividad cada día?
top_hashtags	hashtag (con explode)	usos, engagement promedio	¿Qué hashtags se usan y cuáles rinden más?
ranking_usuarios	usuario	publicaciones, engagement total	¿Quiénes generan más interacción?
actividad_hora	plataforma, hora	publicaciones, engagement promedio	¿A qué hora se publica más?
tipo_contenido	plataforma, tipo	engagement y tasa promedio	¿Funcionan mejor reels, fotos o videos?

Ejemplo: tabla de hashtags (uso de explode)

```
# tabla 2: hashtags mas usados (explode separa la lista)
hashtags = df.select(F.explode("hashtags").alias("hashtag"), "engagement")

top_hashtags = hashtags.groupBy("hashtag").agg(
    F.count("*").alias("usos"),
    F.round(F.avg("engagement"), 2).alias("engagement_prom")
).orderBy(F.desc("usos"))

top_hashtags.write.mode("overwrite").parquet(PREPARED + "/top_hashtags")
```

`mode("overwrite")` permite ejecutar el script varias veces sin error: reemplaza la tabla anterior.

```
python scripts/03_zona_preparada.py
```

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Total publicaciones: 7674
root
 |-- user_id: long (nullable = true)
 |-- comentarios: long (nullable = true)
 |-- compartidos: long (nullable = true)
 |-- fecha_hora: timestamp (nullable = true)
 |-- id_post: string (nullable = true)
 |-- likes: long (nullable = true)
 |-- texto: string (nullable = true)
 |-- tipo_contenido: string (nullable = true)
 |-- vistas: long (nullable = true)
 |-- fecha: date (nullable = true)
 |-- hora: integer (nullable = true)
 |-- hashtags: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- engagement: long (nullable = true)
 |-- tasa_engagement: double (nullable = true)
 |-- nombre_usuario: string (nullable = true)
 |-- pais: string (nullable = true)
 |-- seguidores: integer (nullable = true)
 |-- plataforma: string (nullable = true)
```

Figura 13. Lectura de CLEAN (7.674 registros) y schema. Parquet conserva los tipos: timestamp, date, array, double.

fecha	plataforma	publicaciones	vistas	engagement_prom
2026-09-14	Instagram	260	2704268	839.24
2026-09-14	TikTok	231	5564290	1862.63
2026-09-14	X	277	1058146	304.68
2026-09-15	Instagram	254	2658641	835.66
2026-09-15	TikTok	271	6834969	1934.66
2026-09-15	X	237	927522	311.55
2026-09-16	Instagram	237	2426083	793.51
2026-09-16	TikTok	261	6633200	1901.03
2026-09-16	X	257	1025289	309.83
2026-09-17	Instagram	477	8713762	1471.58
2026-09-17	TikTok	545	24780212	3632.92
2026-09-17	X	512	3744396	563.48
2026-09-18	Instagram	519	8976602	1356.14
2026-09-18	TikTok	469	21474742	3556.75
2026-09-18	X	559	4027111	564.97
2026-09-19	Instagram	516	9363153	1395.34
2026-09-19	TikTok	505	22874720	3565.62
2026-09-19	X	517	3738602	561.86
2026-09-20	Instagram	239	2360400	778.67
2026-09-20	TikTok	265	6334249	1951.75
2026-09-20	X	266	1034306	300.67

Figura 14. Tabla resumen_diario: los días 17, 18 y 19 casi duplican las publicaciones; TikTok concentra la mayor cantidad de vistas.

hashtag	usos	engagement_prom
#santiago	1014	1503.06
#musica	994	1550.51
#valparaiso	992	1482.27
#chile	983	1590.3
#tecnologia	977	1491.55
#deporte	977	1400.26
#comida	972	1466.68
#futbol	968	1532.19
#moda	961	1570.15
#gaming	944	1556.23

only showing top 10 rows

Figura 15. Tabla top_hashtags ordenada por usos: #santiago es el más usado.

user_id	nombre_usuario	pais	seguidores	publicaciones	engagement_total
42	usuario_42	Chile	245	49	83673
221	usuario_221	Chile	324	38	77058
107	usuario_107	Chile	411	32	75471
14	usuario_14	Argentina	217	35	72172
7	usuario_7	Peru	240	35	71184
246	usuario_246	Argentina	278	34	71000
195	usuario_195	Mexico	233	31	67849
33	usuario_33	Colombia	209	35	67737
206	usuario_206	Colombia	1032	30	63923
202	usuario_202	Argentina	312	28	63407

only showing top 10 rows

Figura 16. Tabla ranking_usuarios: los primeros son los que más publican, no los que tienen más seguidores.

plataforma	hora	publicaciones	engagement_prom
Instagram	0	23	920.52
Instagram	1	25	1204.36
Instagram	2	17	1281.76
Instagram	3	18	1321.28
Instagram	4	17	866.88
Instagram	5	18	1261.28
Instagram	6	51	1229.92
Instagram	7	74	1128.03
Instagram	8	101	1096.64
Instagram	9	89	1071.89
Instagram	10	118	1142.36
Instagram	11	131	1223.85
Instagram	12	164	1078.02
Instagram	13	118	1232.19
Instagram	14	123	1214.54
Instagram	15	107	1064.79
Instagram	16	141	1056.23
Instagram	17	151	1260.79
Instagram	18	167	1248.45
Instagram	19	213	1208.14
Instagram	20	233	1171.66
Instagram	21	186	1155.72
Instagram	22	160	1231.04
Instagram	23	57	1281.61

only showing top 24 rows

Figura 17. Tabla actividad_hora (Instagram): el máximo de publicaciones es a las 20:00 h.

plataforma	tipo_contenido	publicaciones	engagement_prom	tasa_prom
TikTok	video	2547	2911.91	7.82
Instagram	reel	632	1212.33	7.81
Instagram	carrusel	608	1194.96	7.89
Instagram	historia	632	1153.83	7.78
Instagram	foto	630	1124.53	7.83
X	texto	875	470.63	7.84
X	foto	859	459.4	7.79
X	video	891	455.91	7.55

Figura 18. Tabla tipo_contenido: TikTok (video) lidera en engagement, pero la tasa es cercana al 7,8% en todos los casos.

```python
# Paso 3: zona preparada, tablas resumen para analisis
import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from config import CLEAN, PREPARED, crear_spark
from pyspark.sql import functions as F

spark = crear_spark("zona_preparada")

# leer datos limpios del paso 2
df = spark.read.parquet(CLEAN + "/publicaciones")
print("Total publicaciones:", df.count())
df.printSchema()

# tabla 1: publicaciones por dia y plataforma
resumen = df.groupBy("fecha", "plataforma").agg(
    F.count("id_post").alias("publicaciones"),
    F.sum("vistas").alias("vistas"),
    F.round(F.avg("engagement"), 2).alias("engagement_prom")
).orderBy("fecha", "plataforma")

resumen.show(21)
resumen.write.mode("overwrite").parquet(PREPARED + "/resumen_diario")

# tabla 2: hashtags mas usados (explode separa la lista)
hashtags = df.select(F.explode("hashtags").alias("hashtag"), "engagement")

top_hashtags = hashtags.groupBy("hashtag").agg(
    F.count("*").alias("usos"),
    F.round(F.avg("engagement"), 2).alias("engagement_prom")
).orderBy(F.desc("usos"))

top_hashtags.show(10)
top_hashtags.write.mode("overwrite").parquet(PREPARED + "/top_hashtags")

# tabla 3: usuarios con mas engagement
ranking = df.groupBy("user_id", "nombre_usuario", "pais", "seguidores").agg(
    F.count("id_post").alias("publicaciones"),
    F.sum("engagement").alias("engagement_total")
).orderBy(F.desc("engagement_total"))

ranking.show(10)
ranking.write.mode("overwrite").parquet(PREPARED + "/ranking_usuarios")

# tabla 4: actividad por hora
por_hora = df.groupBy("plataforma", "hora").agg(
    F.count("id_post").alias("publicaciones"),
    F.round(F.avg("engagement"), 2).alias("engagement_prom")
).orderBy("plataforma", "hora")

por_hora.show(24)
por_hora.write.mode("overwrite").parquet(PREPARED + "/actividad_hora")

# tabla 5: engagement segun tipo de contenido
contenido = df.groupBy("plataforma", "tipo_contenido").agg(
    F.count("id_post").alias("publicaciones"),
    F.round(F.avg("engagement"), 2).alias("engagement_prom"),
    F.round(F.avg("tasa_engagement"), 2).alias("tasa_prom")
).orderBy(F.desc("engagement_prom"))

contenido.show()
contenido.write.mode("overwrite").parquet(PREPARED + "/tipo_contenido")

print("Tablas guardadas en", PREPARED)
spark.stop()
```

Figura 19. Código final de 03_zona_preparada.py con un comentario por cada tabla.

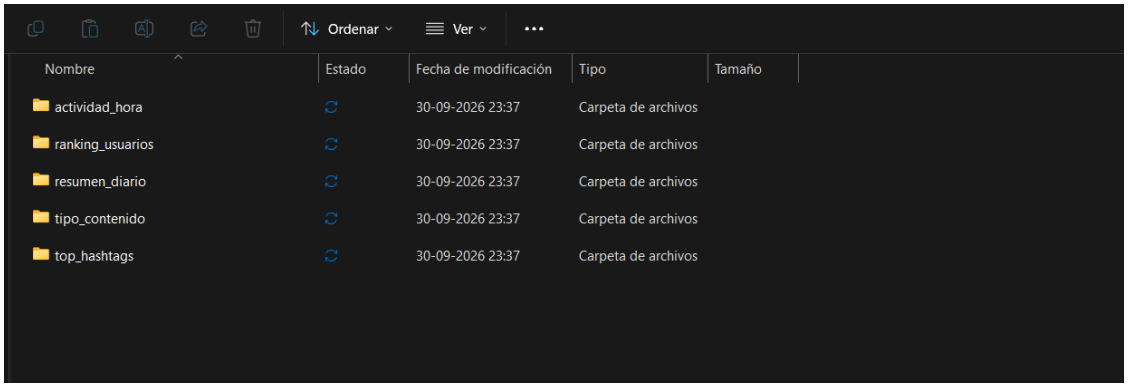


Figura 20. Zona PREPARED con las 5 tablas en Parquet.

11. Paso 4 - Consultas y gráficos

Script: `scripts/04_consultas.py`. Demuestra que el lakehouse se puede consultar de dos maneras sobre los mismos archivos:

- **Spark SQL:** se registra el Parquet como una vista temporal y se consulta con SQL normal.
- **pandas:** lee directamente las tablas Parquet de la zona PREPARED para responder preguntas y graficar.

```
df = spark.read.parquet(CLEAN + "/publicaciones")
df.createOrReplaceTempView("publicaciones")

por_plataforma = spark.sql("""
    SELECT plataforma,
           COUNT(*) AS publicaciones,
           ROUND(AVG(engagement), 2) AS engagement_prom,
           ROUND(AVG(tasa_engagement), 2) AS tasa_prom
    FROM publicaciones
    GROUP BY plataforma
    ORDER BY engagement_prom DESC
""")

python scripts/04_consultas.py
```

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
+----------+-------------+---------------+---------+
|plataforma|publicaciones|engagement_prom|tasa_prom|
+----------+-------------+---------------+---------+
|    TikTok|         2547|        2911.91|     7.82|
| Instagram|         2502|        1171.22|     7.83|
|         X|         2625|         461.96|     7.72|
+----------+-------------+---------------+---------+
```

Figura 21. Consulta Spark SQL directamente sobre archivos Parquet.

```
Plataforma con mas engagement: TikTok
Hashtag mas usado: #santiago
Hashtag con mas engagement: #fiestaspatrias
Hora con mas publicaciones: 20 hrs

Publicaciones por dia:
fecha
2026-09-14     768
2026-09-15     762
2026-09-16     755
2026-09-17    1534
2026-09-18    1547
2026-09-19    1538
2026-09-20     770
Name: publicaciones, dtype: int64
```

Figura 22. Respuestas calculadas con pandas a las preguntas del análisis.

```
Publicaciones por dia:
fecha
2026-09-14     768
2026-09-15     762
2026-09-16     755
2026-09-17    1534
2026-09-18    1547
2026-09-19    1538
2026-09-20     770
Name: publicaciones, dtype: int64
```

Figura 23. Ejecución completa de 04_consultas.py: publicaciones por día (el script termina guardando gráficos y reportes en salida/).

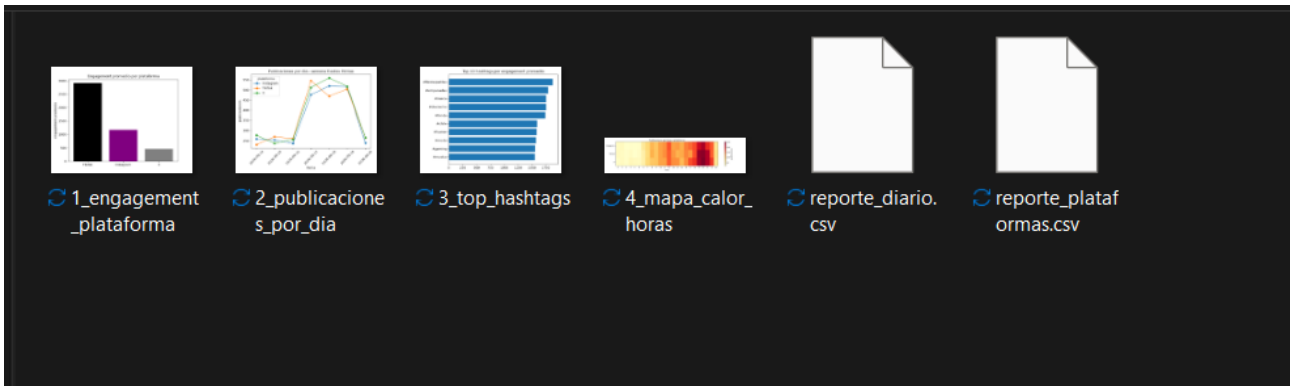


Figura 24. Carpeta salida: 4 gráficos PNG y 2 reportes CSV.

01_ingesta.py 03_zona_preparada.py 04_consultas.py 02_limpieza.py

04_consultas.py > ...

```python
# Paso 4: consultas y graficos sobre la zona preparada
import os
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from config import CLEAN, PREPARED, SALIDA, crear_spark
import pandas as pd
import matplotlib.pyplot as plt

os.makedirs(SALIDA, exist_ok=True)

# consulta con Spark SQL sobre los datos limpios
spark = crear_spark("consultas")
df = spark.read.parquet(CLEAN + "/publicaciones")
df.createOrReplaceTempView("publicaciones")

por_plataforma = spark.sql("""
    SELECT plataforma,
           COUNT(*) AS publicaciones,
           ROUND(AVG(engagement), 2) AS engagement_prom,
           ROUND(AVG(tasa_engagement), 2) AS tasa_prom
    FROM publicaciones
    GROUP BY plataforma
    ORDER BY engagement_prom DESC
""")
por_plataforma.show()

# pasamos el resultado a pandas para graficar
plataformas = por_plataforma.toPandas()
spark.stop()

# leer las tablas de la zona preparada con pandas
resumen = pd.read_parquet(PREPARED + "/resumen_diario")
hashtags = pd.read_parquet(PREPARED + "/top_hashtags")
por_hora = pd.read_parquet(PREPARED + "/actividad_hora")

# respuestas a las preguntas del analisis
print("Plataforma con mas engagement:", plataformas.iloc[0]["plataforma"])
print("Hashtag mas usado:", hashtags.sort_values("usos", ascending=False).iloc[0]["hashtag"])
print("Hashtag con mas engagement:", hashtags.sort_values("engagement_prom", ascending=False).iloc[0]["hashtag"])

total_hora = por_hora.groupby("hora")["publicaciones"].sum()
print("Hora con mas publicaciones:", total_hora.idxmax(), "hrs")

por_dia = resumen.groupby("fecha")["publicaciones"].sum()
print("\nPublicaciones por dia:")
print(por_dia)
```

Figura 25. Código de 04_consultas.py (parte 1): Spark SQL y preguntas con pandas.

```python

# grafico 1: engagement promedio por plataforma
plt.figure()
plt.bar(plataformas["plataforma"], plataformas["engagement_prom"], color=["black", "purple", "gray"])
plt.title("Engagement promedio por plataforma")
plt.ylabel("engagement promedio")
plt.savefig(SALIDA + "/1_engagement_plataforma.png")
plt.close()

# grafico 2: publicaciones por dia (se ve el peak del 18)
tabla_dia = resumen.pivot(index="fecha", columns="plataforma", values="publicaciones")
tabla_dia.plot(marker="o")
plt.title("Publicaciones por dia - semana Fiestas Patrias")
plt.ylabel("publicaciones")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(SALIDA + "/2_publicaciones_por_dia.png")
plt.close()

# grafico 3: hashtags con mas engagement
top10 = hashtags.sort_values("engagement_prom").tail(10)
plt.figure()
plt.barh(top10["hashtag"], top10["engagement_prom"])
plt.title("Top 10 hashtags por engagement promedio")
plt.tight_layout()
plt.savefig(SALIDA + "/3_top_hashtags.png")
plt.close()

# grafico 4: mapa de calor hora x plataforma
calor = por_hora.pivot(index="plataforma", columns="hora", values="publicaciones")
plt.figure(figsize=(12, 3))
plt.imshow(calor, cmap="YlOrRd", aspect="auto")
plt.yticks(range(len(calor.index)), calor.index)
plt.xticks(range(24), range(24))
plt.colorbar(label="publicaciones")
plt.title("Publicaciones por hora y plataforma")
plt.xlabel("hora")
plt.tight_layout()
plt.savefig(SALIDA + "/4_mapa_calor_horas.png")
plt.close()

# reporte final en csv
plataformas.to_csv(SALIDA + "/reporte_plataformas.csv", index=False)
resumen.to_csv(SALIDA + "/reporte_diario.csv", index=False)
print("\nGraficos y reportes guardados en", SALIDA)
```

Figura 26. Código de 04_consultas.py (parte 2): gráficos y reporte CSV.

12. Resultados y hallazgos

Respuestas del análisis

Pregunta	Respuesta
¿Qué plataforma tiene más engagement?	**TikTok** (2.911 promedio) vs Instagram (1.171) y X (462)
¿Cuál es el hashtag más usado?	**#santiago** (1.014 usos)
¿Cuál es el hashtag con más engagement?	**#fiestaspatrias** (1.872 promedio)
¿A qué hora se publica más?	**20:00 h** (rango de 19 a 21 h)
¿Cuánto aumentó la actividad en Fiestas Patrias?	Se **duplicó**: de unas 760 a más de 1.530 publicaciones por día

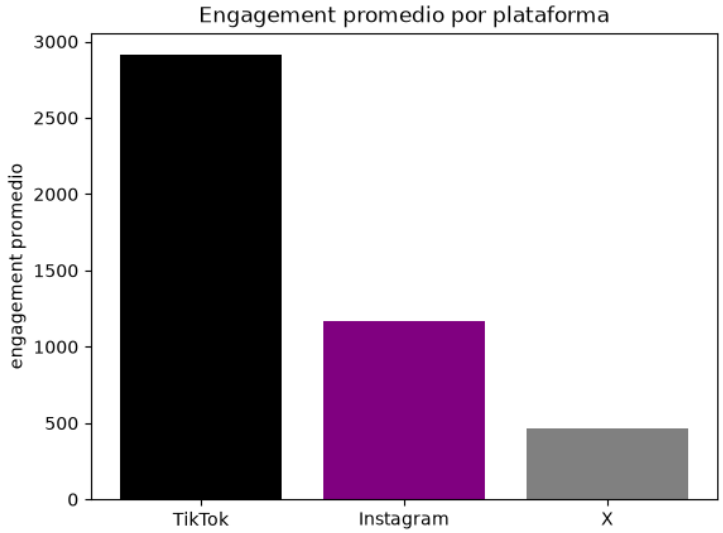


Figura 27. Engagement promedio por plataforma.

Hallazgo 1: el total engaña, la tasa no

TikTok tiene unas 2,5 veces el engagement de Instagram y unas 6 veces el de X. Sin embargo, la **tasa de engagement** (interacciones / vistas) es casi igual en las tres redes, cercana al 7,8%. TikTok no tiene usuarios más "activos": tiene **más vistas**. Por eso conviene medir con tasa y no solo con totales.

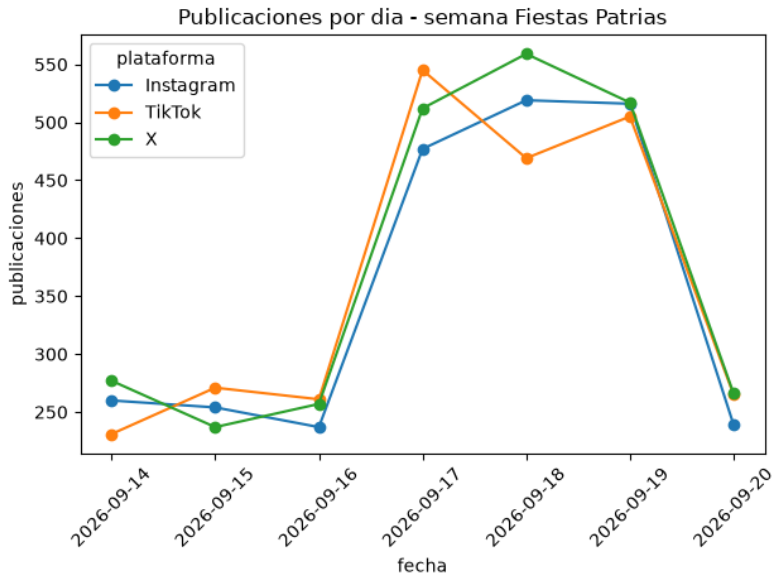


Figura 28. Publicaciones por día y plataforma.

Hallazgo 2: el efecto Fiestas Patrias

Del 17 al 19 de septiembre las publicaciones se duplican en las tres redes, y el 20 vuelven a la normalidad.

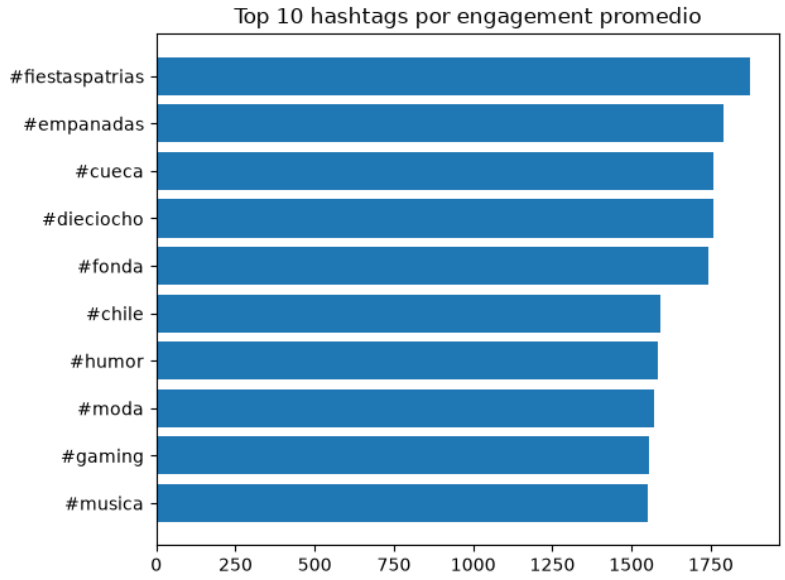


Figura 29. Top 10 hashtags por engagement promedio.

Hallazgo 3: el más usado no es el que más rinde

Los 5 hashtags de Fiestas Patrias (#fiestaspatrias, #empanadas, #cueca, #dieciocho, #fonda) se usaron solo 3 días, pero encabezan el engagement promedio, unos 20% más que los hashtags generales.

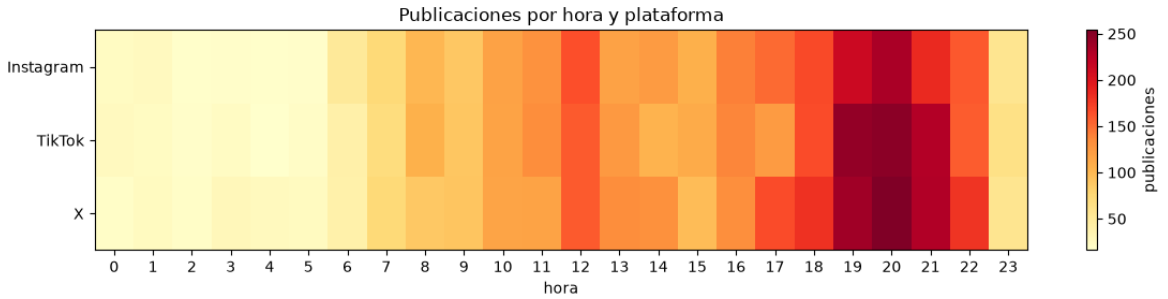


Figura 30. Mapa de calor de publicaciones por hora y plataforma.

Hallazgo 4: la noche concentra la actividad

Mientras más oscuro el color, más publicaciones. El máximo está entre las 19 y las 21 h, con un peak menor al mediodía. De madrugada casi no hay actividad.

Limitaciones (decirlas con honestidad si preguntan)

• Los datos son **simulados**: en el generador, los seguidores no influyen en las vistas, por eso el ranking de usuarios depende de cuánto publican.
• En las horas de madrugada hay muy pocas publicaciones, así que sus promedios de engagement varían mucho por azar y no deben interpretarse como "mejor hora".
• Mejora futura: relacionar seguidores con vistas y consumir una API real.

13. Problemas encontrados y soluciones

#	Problema	Causa	Solución
1	UnsatisfiedLinkError: NativeIO$Windows.access0 al leer/escribir con Spark	Spark (Hadoop) necesita winutils.exe y hadoop.dll en Windows	Descargar la versión 3.3.5, copiar en C:\hadoop\bin, crear HADOOP_HOME y agregar al Path
2	Confusión entre entorno conda y máquina virtual	Concepto nuevo para el equipo	Conda es solo una carpeta con Python y librerías; se activa con conda activate lakehouse
3	ModuleNotFoundError: pyspark (posible)	Entorno no activado	Activar siempre el entorno; verificar que aparezca (lakehouse)
4	Elección de versión de winutils	El repositorio tiene muchas versiones	Usar la que coincide con el Hadoop de Spark (3.3.x), no la 2.6.1
5	No se encontraba "fecha_invalida" al buscar en el JSON	El JSON está en una sola línea muy larga	Formatear en VS Code con Shift + Alt + F y luego Ctrl + F
6	01_ingesta.py borrado por error	Error manual en el explorador	Restaurar desde la Papelera de reciclaje
7	Comentarios desordenados al editar 03	Los números de línea se desplazan al insertar	Ubicar cada comentario justo arriba de su bloque (Alt + flechas en VS Code)
8	Fechas superpuestas en el gráfico por día	Etiquetas largas en el eje X	plt.xticks(rotation=45) y plt.tight_layout()
9	Mensajes "CORRECTO: el proceso con PID... ha sido terminado"	Spark cierra sus procesos de Java	Es normal, no es un error
10	Captura con datos sensibles (pestaña SSH con IP)	Otras pestañas abiertas en el editor	Recortar la imagen antes de ponerla en el informe

14. Recomendaciones y tips

- **Trabajar fuera de OneDrive** (por ejemplo C:\Proyectos): OneDrive sincroniza los muchos archivos que crea Spark y puede volverlo lento o bloquear archivos.
- **Usar una semilla fija** (random.seed) para que los resultados sean reproducibles en cualquier PC.
- **Construir los scripts por partes** y ejecutar después de cada bloque: los errores se encuentran más rápido.
- **Nunca modificar la zona RAW**: si algo sale mal, se regenera CLEAN y PREPARED desde RAW.
- **Usar Parquet en vez de CSV**: pesa unas 5,7 veces menos, guarda los tipos de datos y se lee más rápido.
- **Particionar** por columnas que se usan para filtrar (fecha, plataforma).
- **Medir con tasas** (engagement/vistas) además de totales, para comparar de forma justa.
- **Spark tarda unos segundos en arrancar** (inicia Java): es normal.
- **Para la demo:** ensayar los 4 scripts seguidos (1 a 2 minutos) y tener las capturas de respaldo.

15. Cómo reproducir todo (guía rápida)

```
1) Abrir "Anaconda Prompt (miniconda3)" desde el menu Inicio

2) Solo la primera vez:
   conda create -y -n lakehouse -c conda-forge python=3.11 openjdk=17
                pyspark=3.5 pandas pyarrow matplotlib   (todo en una sola linea)
   (y configurar winutils + HADOOP_HOME como en la seccion 7)

3) Cada vez:
   conda activate lakehouse
   cd "ruta\donde\guardaste\Lakehouse_RedesSociales"

4) Ejecutar en orden:
   python scripts/01_ingesta.py
   python scripts/02_limpieza.py
   python scripts/03_zona_preparada.py
   python scripts/04_consultas.py
```

16. Preguntas probables en la exposición

Pregunta	Respuesta sugerida
¿Por qué Parquet y no CSV?	Es columnar y comprimido: pesó unas 5,7 veces menos que el JSON, conserva los tipos de datos y Spark lee solo las columnas que necesita.
¿Qué diferencia hay entre Lakehouse y Data Lake?	El Data Lake solo guarda archivos. El Lakehouse agrega estructura (zonas, formatos optimizados, schema) y permite consultas SQL como un warehouse.
¿Por qué no Lambda o Kappa?	Nuestro caso es análisis histórico de una semana, no requiere tiempo real ni Kafka. Lakehouse es más simple y suficiente.
¿Qué pasaría con millones de datos?	El mismo código funciona: Spark reparte el trabajo en un clúster y las carpetas serían S3/HDFS. Solo cambian las rutas y el master de Spark.
¿Para qué sirve particionar?	Si se consulta solo TikTok, Spark lee únicamente la carpeta plataforma=TikTok.
¿Qué hace explode?	Convierte una lista (por ejemplo los hashtags de una publicación) en una fila por elemento.
¿Qué es _SUCCESS?	Un archivo vacío que Spark crea al terminar de escribir sin errores.
¿Por qué TikTok tiene más engagement?	Porque tiene más vistas; la tasa de engagement es similar en las tres redes.

17. Glosario

Término	Significado
Data Lakehouse	Arquitectura que combina el almacenamiento en archivos de un Data Lake con las consultas SQL de un Data Warehouse.
RAW / CLEAN / PREPARED	Zonas del lakehouse: datos en bruto, limpios y listos para análisis (también llamadas Bronze / Silver / Gold).
Apache Spark / PySpark	Motor de procesamiento distribuido de datos. PySpark es su versión para Python.
Hadoop / winutils	Hadoop es la base de archivos que usa Spark; winutils son los binarios que necesita para funcionar en Windows.
Parquet	Formato de archivo columnar y comprimido, estándar en Big Data.
Particionado	Guardar los datos en subcarpetas según el valor de una columna (fecha=..., plataforma=...).
Schema	Lista de columnas con su tipo de dato (texto, número, fecha, lista).
DataFrame	Tabla de datos en memoria (en Spark o en pandas).
groupBy / agg	Agrupar filas por una o más columnas y calcular conteos, sumas o promedios.
explode	Función que separa una lista en varias filas.
Engagement	Interacciones totales: likes + comentarios + compartidos.
Tasa de engagement	Engagement dividido por las vistas, en porcentaje.
Variable de entorno	Dato que Windows guarda para que los programas lo lean (por ejemplo, HADOOP_HOME).
Entorno conda	Carpeta aislada con su propio Python y librerías. No es una máquina virtual.

18. Índice de evidencias (carpeta evidencias/)

Carpeta	Archivo
00_ingesta	01_consola_ingesta.png
00_ingesta	02_carpeta_raw_publicaciones.png
00_ingesta	04_json_fecha_invalida.png
00_ingesta	05_csv_usuarios.png
01_instalacion_winutils	01_carpeta_hadoop_bin.png
01_instalacion_winutils	02_propiedades_del_sistema.png
01_instalacion_winutils	03_variables_entorno_hadoop_home.png
01_instalacion_winutils	04_setx_hadoop_home.png
02_limpieza	01_winutils_ok_y_limpieza_antes.png
02_limpieza	02_limpieza_despues.png
02_limpieza	03_carpeta_clean_particiones.png
02_limpieza	04_archivos_parquet.png
03_zona_preparada	01_lectura_clean_schema.png
03_zona_preparada	02_tabla_resumen_diario.png
03_zona_preparada	03_tabla_top_hashtags.png
03_zona_preparada	04_tabla_ranking_usuarios.png
03_zona_preparada	05_tabla_actividad_hora.png
03_zona_preparada	06_tabla_tipo_contenido.png
03_zona_preparada	07_codigo_zona_preparada.png
03_zona_preparada	08_carpeta_prepared.png
04_consultas	01_consulta_spark_sql.png
04_consultas	02_respuestas_pandas.png
04_consultas	03_graficos_guardados.png
04_consultas	04_carpeta_salida.png
04_consultas	05_codigo_consultas_parte1.png
04_consultas	06_codigo_consultas_parte2.png
salida	1_engagement_plataforma.png
salida	2_publicaciones_por_dia.png
salida	3_top_hashtags.png
salida	4_mapa_calor_horas.png
salida	reporte_diario.csv
salida	reporte_plataformas.csv

Nota: la captura 00_ingesta/05_csv_usuarios.png se recortó en esta bitácora para ocultar las pestañas del editor. Antes de usarla en el informe, recortar también la parte superior.